GBIN5U03 - La mémoire

Guillaume Huard

Université Grenoble Alpes

Plan

Organisation de la mémoire

Pointeurs

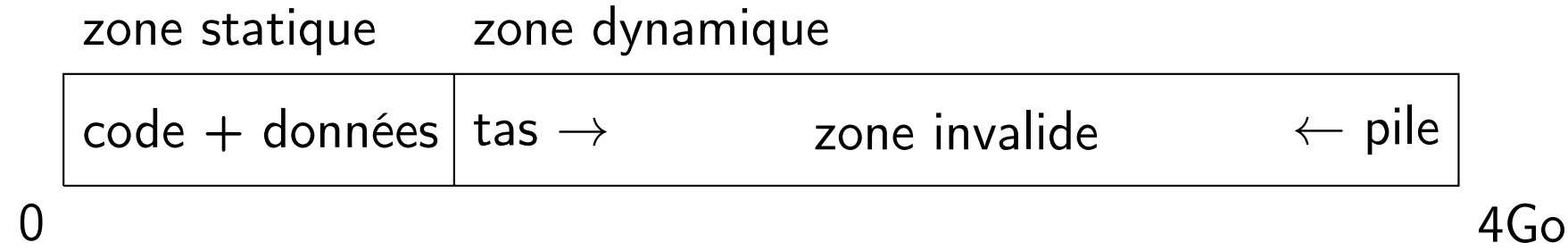
Allocation dynamique

Arithmétique sur les pointeurs

Organisation en mémoire d'un programme

L'OS fait en sorte qu'à l'exécution, un programme

- voie tout l'espace d'adressage comme dédié à son exécution
- soit chargé en mémoire selon le schéma suivant



On y distingue les zones suivantes

- la zone statique où se trouvent les sections du programme : `.text`, `.data`, `.bss`
- le tas, dans lequel de la mémoire peut être allouée sur demande
- la pile, servant à gérer les appels de fonctions

Pile

Sert à gérer les appels de fonction, on y trouve

- l'adresse de retour (instruction suivant l'appel)
- les paramètres formels, initialisés aux valeurs des paramètres effectifs (passage par valeur)
- les variables locales à la fonction

Rend possible l'implémentation de la récursivité

```
void func(int n) {      // une copie de la valeur de n
        if (n > 0)          // est empilée à chaque appel
                func(n-1);
        printf("%d\n", n); // l'affichage se produit après
}                          // la série d'appels, mais toutes
int main() {               // les valeurs sont dans la pile
        func(10);
        return 0;
}
```

Écrit explicitement en assembleur par le compilateur

Pile explicite si on tranforme du récursif en itératif

```c
void func(int n) {
        if (n > 0)
                func(n-1);
        printf("%d\n", n);
}

int main() {
        func(10);
        return 0;
}
```

$\Longleftrightarrow$

```c
int n=10;
pile p;

p = pile_vide();
while (n >= 0) {
        // Style typique du C :
        // Adresse explicite
        empiler(n, &p);
        n--;
}
while (!est_pile_vide(p)) {
        n = depiler(&p);
        printf("%d\n", n);
}
detruire_pile(p);
```

Le plus compliqué est de coder l'état de retour d'un appel en structure de données dans une pile

Une fonction ne peut pas retourner un tableau

Lié à l'allocation dans la pile des tableaux locaux

- les tableaux sont manipulés par adresse
- les variables locales sont libérées en fin de fonction

```
int[10] lire_tableau() { // (syntaxe invalide)
        int t[10];       // tableau alloué dans la pile

        for (int i=0; i<10; i++)
                scanf(" %d", &t[i]);
        return t;
}                        // tableau libéré :
                         // - le pointeur de pile est déplacé
int main() {             // - les valeurs seront écrasées au
        int t[10];       // prochain appel de fonction

        t = lire_tableau();  // (syntaxe invalide)
        for (int i=0; i<10; i++)
                printf("%d\n", t[i]);
        return 0;
}
```

Une fonction peut retourner une structure

Les structures sont passées par copie

```
typedef struct {
        int elt[10];     // champ ayant pour taille 10*int
} tableau;

tableau lire_tableau() {
        tableau t;          // structure allouée dans la pile

        for (int i=0; i<10; i++)
                scanf(" %d", &t.elt[i]);
        return t;
}                          // structure passée par copie :
int main() {              // tous ses champs sont copiés
        tableau t;

        t = lire_tableau();
        for (int i=0; i<10; i++)
                printf("%d\n", t.elt[i]);
        return 0;
}
```

La pile n'est pas infinie

Le système alloue une pile de taille fixe, en général assez grande

- un débordement de pile est souvent une récursion infinie
- attention aux fonctions qui y allouent beaucoup de données

```
#define TAILLE ((int) (1024/sizeof(int)))
void func(int n) {
        int t[TAILLE];
        t[0]=0;
        for (int i=1; i<TAILLE; i++)
                t[i] = i+t[i-1];
        if (n > 1)
                func(n-1);
}
int main() {
        for (int i=1; 1; i*=2) {
                printf("Essai pour %dKo\n", i);
                func(i);
        }
        return 0;
}
```

Mauvaises habitudes

Le *buffer overflow* est une attaque exploitant

- l'utilisation commune de `scanf("%s",...)` et `gets(...)`
- l'organisation du programme en mémoire

Principe avec la pile :

```
... func (...) {
        char s[X];
        scanf ("%s", s);
        ...
```

Une entrée trop longue va déborder sur ce qui est stocké après le tableau dans la pile et finir par écraser l'adresse de retour avec

- l'adresse d'un code malicieux placé dans la chaine en entrée
  - l'organisation de l'espage d'adressage doit être connue
  - la zone mémoire réservée à la pile doit être exécutable

  Ex : *crack* de la Wii, Microsoft SQL Server 2000
- une valeur invalide : erreur au retour de la fonction

static

Le mot clé `static`

- limite la portée d'un symbole à l'unité de compilation courante
- alloue le symbole à une position fixe dans la zone statique

```
static FILE *f;

void init() {
        static int first=1;
        if (first) {
                f = fopen("sortie.txt", "w");
                first = 0;
        }
}
int main() {
        init();
        init();
        fprintf(f, "Salut\n");
        fclose(f);
        return 0;
}
```

autres mots clé

`register` :
demande au compilateur de placer la variable dans un registre

```
void echange(int *a, int *b) {
    register int tmp;
    tmp = *a;
    *a = *b;
    *b = tmp;
}
```

`volatile` :
informe le compilateur que le contenu de la variable peut changer en raison d'une cause extérieure au programme

- programmation multithreadée
- registres d'entrée / sortie

Plan

Organisation de la mémoire

Pointeurs

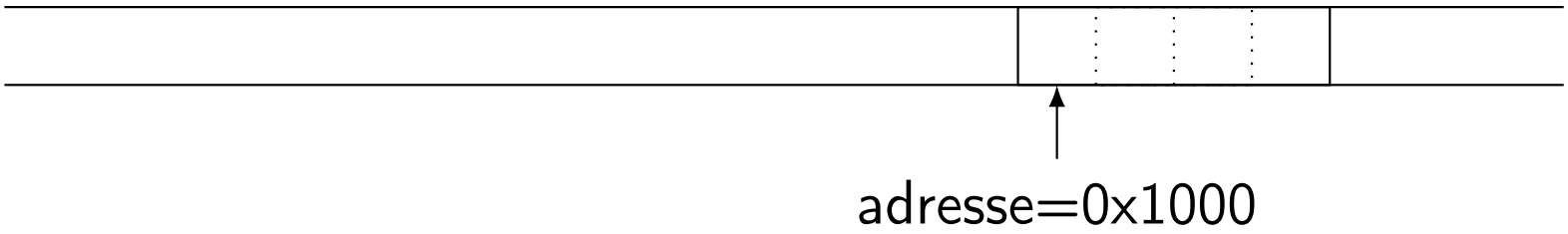
Allocation dynamique

Arithmétique sur les pointeurs

Adresse et stockage en mémoire

Un entier de type `int` occupe généralement 4 octets

- ces octets sont contigus en mémoire
- l'adresse de l'entier est l'adresse de son premier octet



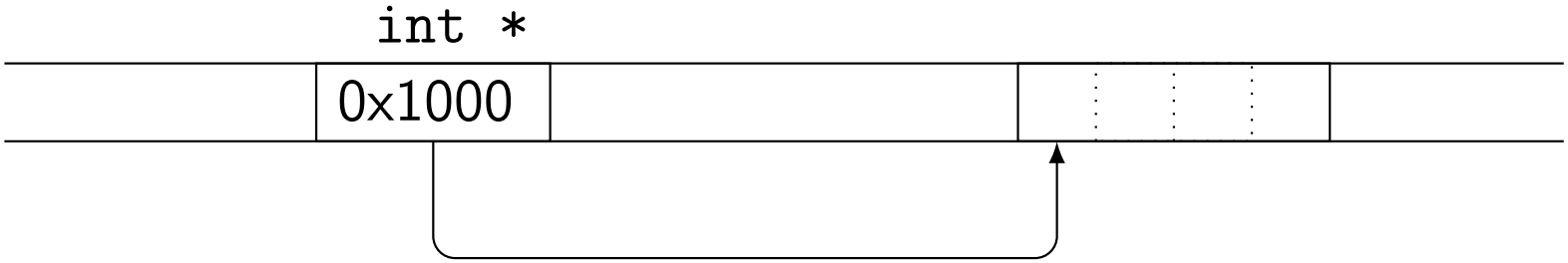
L'accès à une variable stockée en mémoire dépend de sa taille

- en assembleur, multiples instructions, ex : `ldr`, `ldrh`, `ldrb`
- en C, taille de l'accès déterminée par le type

Pointeur

Un pointeur est la combinaison

- d'une valeur correspondant à une adresse
- d'un type donnant la taille de l'objet stocké à cette adresse




```c
int main() {
        int i=41;
        int *p;    // déclaration d'un pointeur :
                   //    type pointé suivi d'une *
        p = &i;    // opérateur & pour référencer :
                   //    récupère un pointeur sur l'opérande
        *p += 1;   // opérateur * pour déréférencer :
                   //    récupère l'objet pointé en mémoire
        printf("%d\n", i);
        return 0;
}
```

Cast

L'opérateur de *cast* permet de changer le type d'une expression
Pour un pointeur, cela peut changer la taille des accès

```
int main() {
        int i=42;
        char *p;

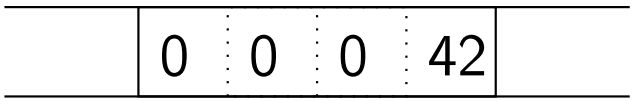
        p = (char *) &i;
        printf("%d\n", *p);
        return 0;
}
```

Cast

L'opérateur de *cast* permet de changer le type d'une expression
Pour un pointeur, cela peut changer la taille des accès

```
int main() {
        int i=42;
        char *p;

        p = (char *) &i;
        printf("%d\n", *p);
        return 0;
}
```



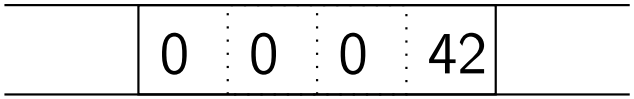
big endian

Cast

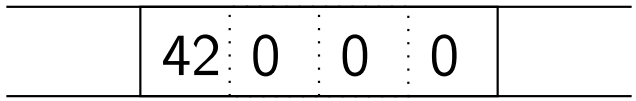
L'opérateur de *cast* permet de changer le type d'une expression
Pour un pointeur, cela peut changer la taille des accès

```
int main() {
        int i=42;
        char *p;

        p = (char *) &i;
        printf("%d\n", *p);
        return 0;
}
```



big endian



little endian

Un Type particulier

Le type `void *` est un type de pointeur dit générique :

- correspond à une adresse, sans idée sur ce qui y est stocké
- ne peut pas être déréférencé

$\Rightarrow$ on le *cast* avant de le déréférencer

`NULL` est un pointeur, de type `void *`, dont l'adresse est invalide

Plan

Organisation de la mémoire

Pointeurs

Allocation dynamique

Arithmétique sur les pointeurs

Allocation dans le tas

Fourni par le système :

- `void * malloc(size_t size);`
  alloue un bloc contigu de `size` octets dans le tas
  renvoie son adresse ou `NULL` si cette allocation échoue
- `void free(void *ptr);`
  libère un bloc précédemment alloué par `malloc`

```c
int main() {
        int *p;
        p = malloc(sizeof(int));
        if (p == NULL) exit(1);
        *p = 42;
        printf("%d\n", *p);
        free(p);
        return 0;
}
```

ATTENTION : ne pas libérer un bloc alloué est une erreur
$\Rightarrow$ risque de fuite mémoire

Et les structures ?

De manière analogue, on peut allouer des structures

```
typedef struct {
        double r, i;
} complexe;

int main() {
        complexe *p;
        p = malloc(sizeof(complexe));
        if (p == NULL)
                exit(1);

        (*p).r = 4.0;
        (*p).i = 2.4;
        // Le C offre aussi un opérateur '->'
        // équivalent à '(*).'
        p->r = 4.0;
        p->i = 2.4;
        return 0;
}
```

Exemple, une pile, entête

```
#ifndef __PILE_H__
#define __PILE_H__

struct noeud {
        int donnee;
        struct noeud *prochain;
};
typedef struct noeud *pile;

pile pile_vide();
int est_pile_vide(pile p);
void empiler(int donnee, pile *p);
int depiler(pile *p);
void detruire_pile(pile p);

#endif
```

Exemple, une pile, implémentation

```
pile pile_vide() {
        return NULL;
}
int est_pile_vide(pile p) {
        return p == NULL;
}
void empiler(int donnee,
                        pile *p) {
        pile nouveau = malloc(
                sizeof(struct noeud));
        assert(nouveau != NULL);
        nouveau->donnee = donnee;
        nouveau->prochain = *p;
        *p = nouveau;
}
```

```
int depiler(pile *p) {
        pile prochain =
                (*p)->prochain;
        int resultat =
                (*p)->donnee;
        free(*p);
        *p = prochain;
        return resultat;
}
void detruire_pile(pile p) {
        while (p) {
                pile prochain =
                        p->prochain
                free(p);
                p = prochain;
        }
}
```

Exemple, une pile, programme principal

```
        p = pile_vide();
        while (n >= 0) {
                // Style typique du C :
                // Adresse explicite
                empiler(n, &p);
                n--;
        }
        while (!est_pile_vide(p)) {
                n = depiler(&p);
                printf("%d\n", n);
        }
        detruire_pile(p);
        return 0;
}
```

Exemple, une pile, programme principal plus abstrait

```
int n=10;
pile p;

p = pile_vide();
while (n >= 0) {
        // Version plus abstraite
        // (similaire au modèle java) :
        // On cache le passage par adresse
        empiler(n, p);
        n--;
}
while (!est_pile_vide(p)) {
        n = depiler(p);
        printf("%d\n", n);
}
detruire_pile(p);
```

Exemple, une pile, entête plus abstrait

```
#ifndef __PILE_ABS_H__
#define __PILE_ABS_H__

struct noeud {
        int donnee;
        struct noeud *prochain;
};
typedef struct donnees_pile { // Une indirection de plus :
        struct noeud *sommet;       // - pointeur vers le sommet
} *pile;                            // - adresse de ce pointeur

pile pile_vide();
int est_pile_vide(pile p);
void empiler(int donnee, pile p);
int depiler(pile p);
void detruire_pile(pile p);

#endif
```

Exemple, une pile, implémentation plus abstraite

```
pile pile_vide() {    // Et ça coûte un malloc en plus
        pile resultat = malloc(sizeof(struct donnees_pile));
        assert(resultat != NULL);
    resultat->sommet = NULL;
    return resultat;
}
int est_pile_vide(pile p) {
    return p->sommet == NULL;
}
void empiler(int donnee, pile p) {
        struct noeud *nouveau = malloc(sizeof(struct noeud));
        assert(nouveau != NULL);
    nouveau->donnee = donnee;
    nouveau->prochain = p->sommet;
    p->sommet = nouveau;
}
```

Exemple, une pile, implémentation plus abstraite (suite)

```
int depiler(pile p) {
        struct noeud *prochain = p->sommet->prochain;
        int resultat = p->sommet->donnee;
    free(p->sommet);
    p->sommet = prochain;
    return resultat;
}
void detruire_pile(pile p) {
        while (p->sommet) {
                struct noeud *prochain = p->sommet->prochain;
                free(p->sommet);
        p->sommet = prochain;
        }
        free(p);
}
```

Plan

Organisation de la mémoire

Pointeurs

Allocation dynamique

Arithmétique sur les pointeurs

Arithmétique sur les pointeurs

Limité à quelques opérations

- addition d'une valeur entière n à un pointeur p :
  pointeur situé n éléments plus loin en mémoire (son adresse dépend donc du type du pointeur en plus de n)
- soustraction d'une valeur entière n à un pointeur p : analogue
- différence entre deux pointeurs p et q :
  valeur entière à ajouter à q pour obtenir p

```
int main() {
        int *p;
        p = malloc(sizeof(int)*10);
        if (p == NULL)
                exit(1);
        for (int i=0; i<10; i++)
                *(p+i) = 42;
        free(p);
        return 0;
}
```

Equivalence pointeurs tableaux

Les tableaux sont manipulés par adresse

- pointeur constant sur le premier élément
- `t[i]` $\Longleftrightarrow$ `*(t+i)`

On peut donc se servir du résultat d'un malloc comme d'un tableau

```
int main() {
        int *p;
        p = malloc(sizeof(int)*10);
        if (p == NULL)
                exit(1);

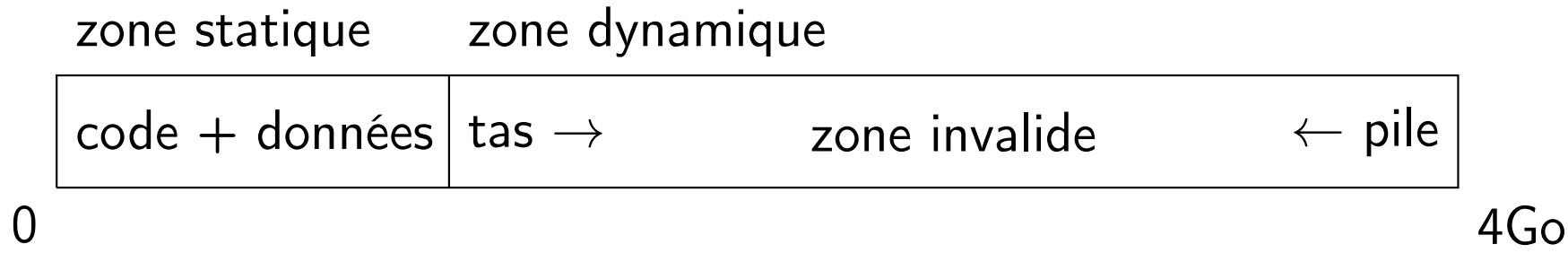
        for (int i=0; i<10; i++)
                p[i] = 42;
        free(p);
        return 0;
}
```

Tableau multidimensionnels

On ne peut pas allouer (simplement) avec un seul `malloc` :

```
int main() {
        int **p, ok=1;
        p = malloc(sizeof(int *)*10); // tableau de pointeurs
        if (p == NULL)
                exit(1);
        for (int i=0; i<10 && ok; i++) {
                p[i] = malloc(sizeof(int)*10); // une allocation
                ok = (p[i] != NULL);           // par ligne
        }
        if (ok) {
                for (int i=0; i<10; i++)
                        for (int j=0; j<10; j++)
                                p[i][j] = 42;
        } // Attention à ne pas libérer de pointeur invalide
        for (int i=0; i<10 && (p[i] != NULL); i++)
                free(p[i]);
        free(p);
        return 0;
}
```

Effets des erreurs mémoire



Accès totalement invalide (pointeur NULL, gros débordement)

- Segmentation Fault (signal 11)

Accès légèrement en dehors d'un tableau

- corruption des structures de l'allocateur mémoire (vu par glibc)
- modification de la valeur d'autres variables

Fuites mémoire

- utilisation excessives des ressources mémoire, swap

Valgrind

Vérifie la bonne utilisation des blocs alloués dans le tas

- vérifie que les accès restent dans des blocs alloués
- vérifie que tous les blocs alloués sont libérés

Utilisation simple

```
valgrind ma_commande et ses arguments
```

Mais `valgrind` sait faire beaucoup d'autre choses

- *profiling* du cache, du prédicteur de branchements, du tas
- détection d'erreurs de programmation multi-threadée
- autre outils expérimentaux d'analyse de code

Alternative `clang` *address sanitizer*

- nécessite de recompiler avec
  - `-fsanitize=address` à toutes les étapes de compilation
  - messages plus intuitifs : `-O1`, `-fno-omit-frame-pointer`

Exemple erroné

```
int main() {
        int **p, ok=1;
        p = malloc(sizeof(int *)*10);
        if (p == NULL)
                exit(1);
        for (int i=0; i<10 && ok; i++) {
                p[i] = malloc(sizeof(int)*10);
                ok = (p[i] != NULL);
        }
        if (ok) {
                for (int i=0; i<10; i++)
                        for (int j=0; j<=10; j++)
                                p[i][j] = 42;
        }
        //for (int i=0; i<10 && (p[i] != NULL); i++)
        //    free(p[i]);
        //free(p);
        return 0;
}
```

Affichage de valgrind

```
==5252== Memcheck, a memory error detector
==5252== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.
==5252== Using Valgrind-3.14.0.GIT and LibVEX; rerun with -h for copyright info
==5252== Command: ./ex_tableau_erreur
==5252==
==5252== Invalid write of size 4
==5252==    at 0x100000F48: main (ex_tableau_erreur.c:16)
==5252==  Address 0x100ded7b8 is 0 bytes after a block of size 40 alloc'd
==5252==    at 0x1000ACAB6: malloc (vg_replace_malloc.c:302)
==5252==    by 0x100000ED8: main (ex_tableau_erreur.c:10)
==5252==
==5252==
==5252== HEAP SUMMARY:
==5252==     in use at exit: 18,781 bytes in 171 blocks
==5252==   total heap usage: 192 allocs, 21 frees, 27,229 bytes allocated
==5252==
==5252== LEAK SUMMARY:
==5252==    definitely lost: 488 bytes in 9 blocks
==5252==    indirectly lost: 7,288 bytes in 18 blocks
==5252==      possibly lost: 72 bytes in 3 blocks
==5252==    still reachable: 32 bytes in 1 blocks
==5252==         suppressed: 10,901 bytes in 140 blocks
==5252== Rerun with --leak-check=full to see details of leaked memory
```

Exemple

```
NAME
     memcpy -- copy memory area

LIBRARY
     Standard C Library (libc, -lc)

SYNOPSIS
     #include <string.h>

     void *
     memcpy(void *restrict dst, const void *restrict src, size_t n);

DESCRIPTION
     The memcpy() function copies n bytes from memory area
     src to memory area dst.  If dst and src overlap,
     behavior is undefined.  Applications in which dst and
     src might overlap should use memmove(3) instead.

RETURN VALUES
     The memcpy() function returns the original value of dst.
```

Deux propositions

Une fausse

```
void *my_memcpy(void *dst, void *src, size_t n) {
        for (int i=0; i<n; i++)
                *dst++ = *src++;
        return dst;
}
```

Deux propositions

Une fausse

```
void *my_memcpy(void *dst, void *src, size_t n) {
        for (int i=0; i<n; i++)
                *dst++ = *src++;
        return dst;
}
```

Une correcte

```
void *my_memcpy(void *dst, void *src, size_t n) {
        char *d;
        char *s;

        d = dst;
        s = src;
        for (int i=0; i<n; i++)
                *d++ = *s++;
        return dst;
}
```

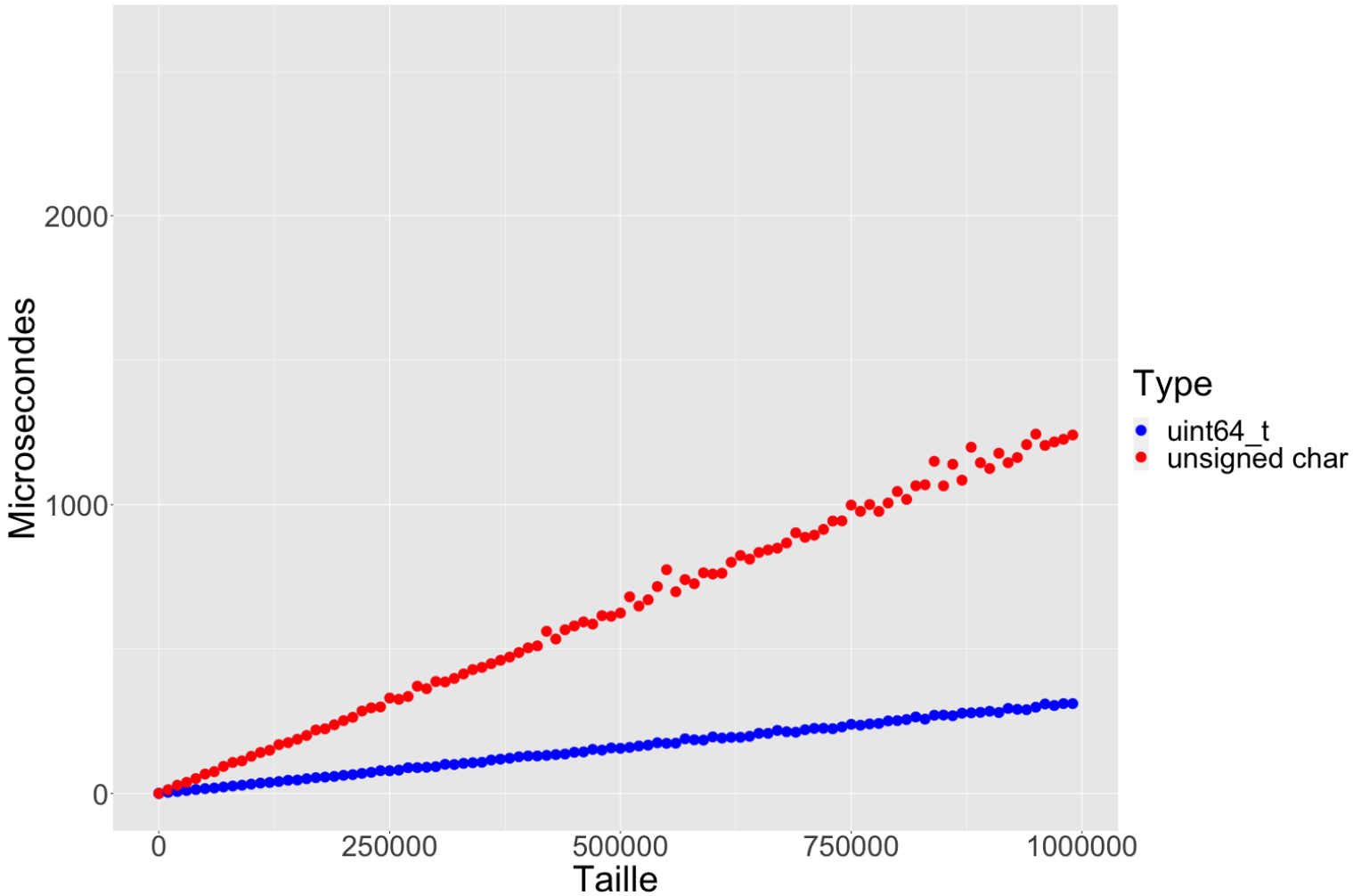
Opérations mémoire plus grosses

```
typedef uint64_t word;

void *my_memcpy(void *dst, void *src, size_t n) {
        word *b; word *a;
        size_t c;
        char *d; char *s;

        b = dst;
        a = src;
        c = n / sizeof(word);
        for (int i=0; i<c; i++)
                *b++ = *a++;
        d = (char *) b;
        s = (char *) a;
        n = n % sizeof(word);
        for (int i=0; i<n; i++)
                *d++ = *s++;
        return dst;
}
```

Effet sur le temps d'exécution



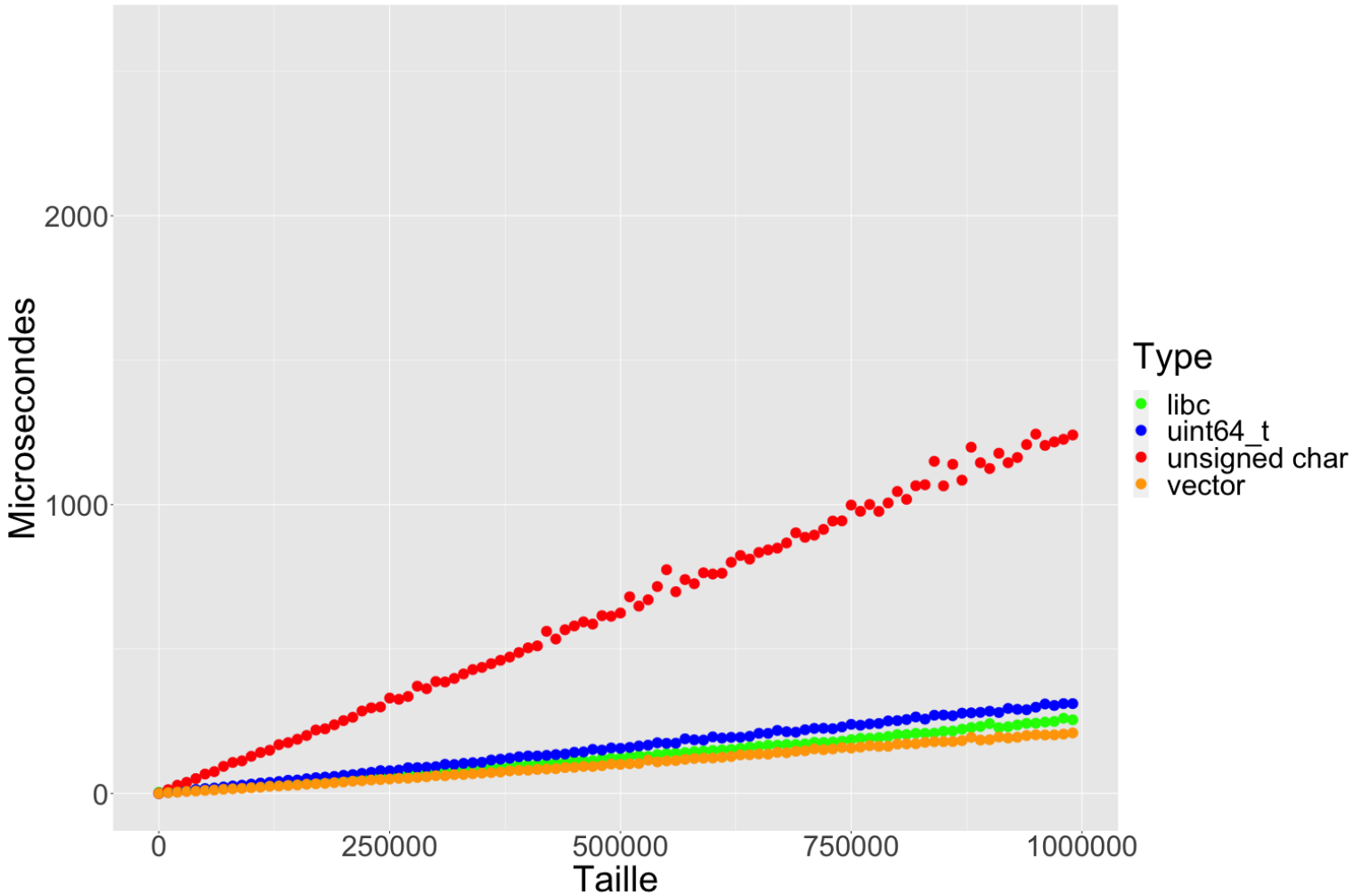
sur un Core I5-1038NG7 avec 16Go de RAM et OSX 10.15.6

Opérations mémoire encore plus grosses

```
// Extension hors standard
typedef uint64_t vec __attribute__ ((ext_vector_type(16)));

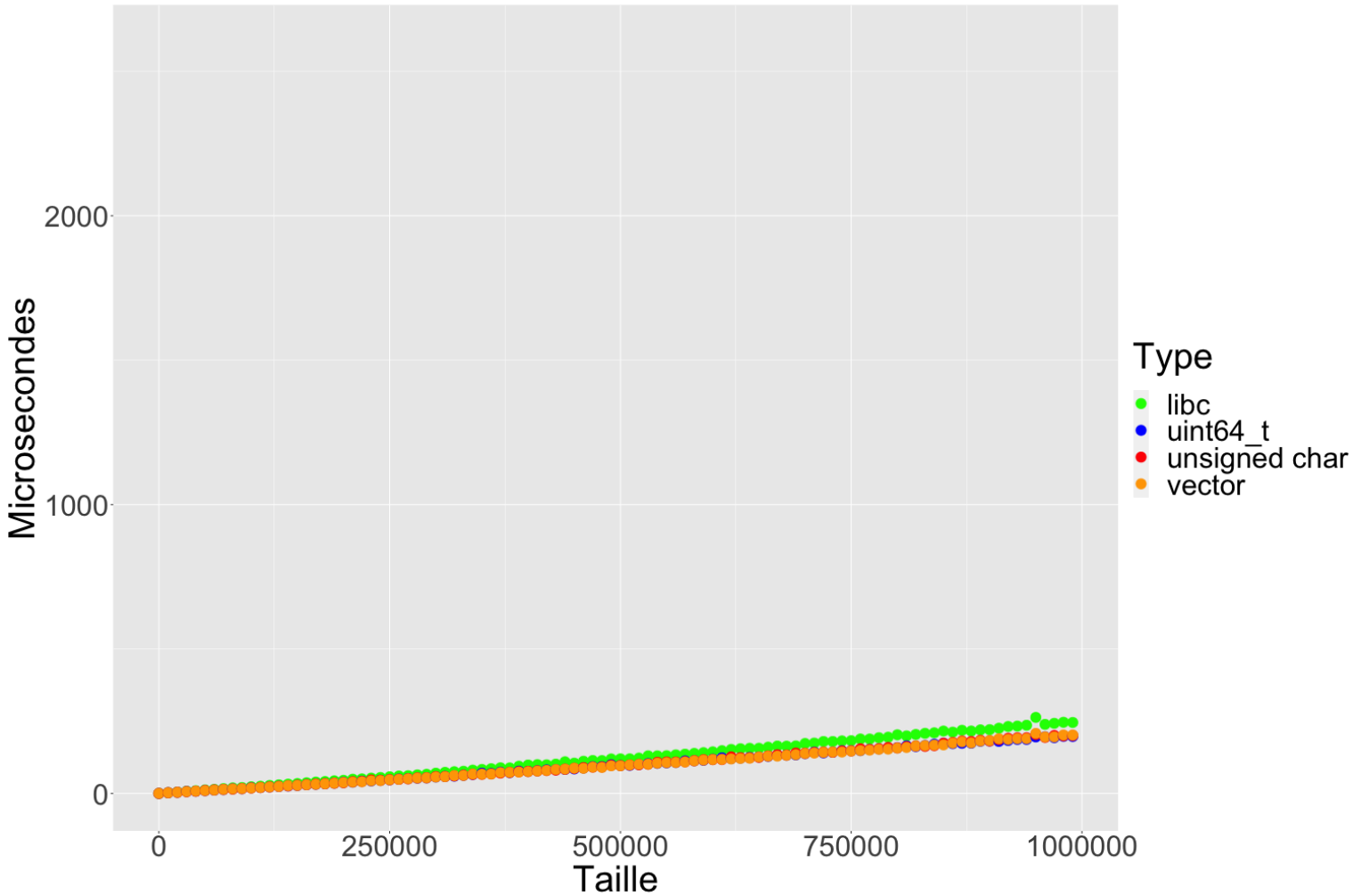
void *my_memcpy(void *dst, void *src, size_t n) {
        vec *b; vec *a;
        size_t c;
        char *d; char *s;
        b = dst;
        a = src;
        c = n / sizeof(vec);
        for (int i=0; i<c; i++) {
                *b++ = *a++;
        }
        d = (char *) b;
        s = (char *) a;
        n = n % sizeof(vec);
        for (int i=0; i<n; i++)
                *d++ = *s++;
        return dst;
}
```

Effet sur le temps d'exécution



sur un Core I5-1038NG7 avec 16Go de RAM et OSX 10.15.6

En compilant avec -O2...



sur un Core I5-1038NG7 avec 16Go de RAM et OSX 10.15.6

Conclusion

Différentes zones mémoire pour différents usages

- pile : variables locales
- tas : blocs alloués avec `malloc`

Compilateur et `valgrind` pour aider avec la mémoire

- risques de débordement
- mauvais usages (`gets`, `scanf("%s")`)

Ne pas viser la performance en premier

- la correction est plus importante
- penser à -O2 (entre autres)
- utiliser un *profiler* pour cibler les parties inefficaces

Questions ?